TeeTime Social Golf App

Development Session Report

December 14, 2024

Contents

1 Session Overview

This document summarizes the development work completed during the December 14, 2024 session for the TeeTime Social Golf App. The session focused on fixing critical API errors, improving UI/UX for the OTP verification flow, and ensuring proper route configuration across the backend.

1.1 Key Accomplishments

- Fixed Create Invite API error with Twilio integration
- Resolved OTP input visibility issue when keyboard is open
- Fixed multiple API failures across Friends, Roster, and Courses endpoints
- Corrected Express route ordering to prevent route conflicts
- Added proper method bindings to all controller classes

2 Bug Fixes and Improvements

2.1 Create Invite API Error

2.1.1 Problem

The create invite endpoint was failing due to multiple issues:

1. The `this` context was lost when controller methods called internal helper methods
2. Twilio client was initialized with empty `TWILIO_AUTH_TOKEN`
3. The `response_at` column referenced in queries didn't exist in the schema

2.1.2 Solution

File: `backend/controllers/inviteController.js`

```
// Added constructor with method bindings
constructor() {
    this.createInvites = this.createInvites.bind(this);
    this.createFromRoster = this.createFromRoster.bind(this);
    this.getPendingInvites = this.getPendingInvites.bind(this);
    this.getSentInvites = this.getSentInvites.bind(this);
    this.respondToInvite = this.respondToInvite.bind(this);
    this.getInviteResponses = this.getInviteResponses.bind(this);
    this.getUserResponseHistory = this.getUserResponseHistory.bind(this
        );
    this.cancelInvite = this.cancelInvite.bind(this);
    this.sendInviteSms = this.sendInviteSms.bind(this);
    this.getWaitlist = this.getWaitlist.bind(this);
}
```

```
// Fixed Twilio initialization to use API Key credentials
if (process.env.TWILIO_ACCOUNT_SID && process.env.TWILIO_SID
    && process.env.TWILIO_SECRET) {
    twilioClient = twilio(process.env.TWILIO_SID,
        process.env.TWILIO_SECRET, {
        accountSid: process.env.TWILIO_ACCOUNT_SID
```

```
    });
}
```

File: `backend/repositories/inviteRepository.js`

```
-- Changed from non-existent column
SET status = $1, response_at = NOW()
-- To existing column
SET status = $1, updated_at = NOW()
```

2.2 OTP Input Keyboard Visibility

2.2.1 Problem

When users typed in the OTP field on the login screen, the input section became blank/hidden when the keyboard opened. The content only appeared after closing the keyboard.

2.2.2 Solution

File: `TTSGolfApp/screens/LoginScreen.tsx`

Added `ScrollView` import and wrapped card content:

```
<KeyboardAvoidingView
    style={styles.cardWrapper}
    behavior={Platform.OS === "ios" ? "padding" : "height"}
    keyboardVerticalOffset={Platform.OS === "ios" ? 0 : 20}
>
    <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
        <ScrollView
            style={styles.card}
            contentContainerStyle={styles.cardScrollContent}
            keyboardShouldPersistTaps="handled"
            showsVerticalScrollIndicator={false}
            bounces={false}
        >
            {/* Card content */}
        </ScrollView>
    </TouchableWithoutFeedback>
</KeyboardAvoidingView>
```

File: `TTSGolfApp/theme/styles.ts`

Added new style for scroll content:

```
cardScrollContent: {
    flexGrow: 1,
    paddingHorizontal: spacing.xl,
    paddingTop: spacing.xxl,
    paddingBottom: spacing.lg,
}
```

2.3 Controller Method Binding Fixes

2.3.1 Problem

Multiple controllers were missing constructor bindings, causing `this` context to be lost when Express passed methods as route handlers.

2.3.2 Solution

File: `backend/controllers/teeTimeController.js`

```
constructor() {
    this.createTeeTime = this.createTeeTime.bind(this);
    this.getTeeTimes = this.getTeeTimes.bind(this);
    this.getUpcoming = this.getUpcoming.bind(this);
    this.getTeeTime = this.getTeeTime.bind(this);
    this.cancelTeeTime = this.cancelTeeTime.bind(this);
    this.getCalendar = this.getCalendar.bind(this);
}
```

File: `backend/controllers/rosterController.js`

```
constructor() {
    this.getRoster = this.getRoster.bind(this);
    this.getOnDeck = this.getOnDeck.bind(this);
    this.addToRoster = this.addToRoster.bind(this);
    this.removeFromRoster = this.removeFromRoster.bind(this);
    this.reorderRoster = this.reorderRoster.bind(this);
    this.toggleOnDeck = this.toggleOnDeck.bind(this);
    this.searchUsers = this.searchUsers.bind(this);
    this.getGroups = this.getGroups.bind(this);
    this.createGroup = this.createGroup.bind(this);
    this.updateGroup = this.updateGroup.bind(this);
    this.deleteGroup = this.deleteGroup.bind(this);
    this.getGroupMembers = this.getGroupMembers.bind(this);
    this.updateMemberGroup = this.updateMemberGroup.bind(this);
    this.moveMember = this.moveMember.bind(this);
    this.getMemberProfile = this.getMemberProfile.bind(this);
    this.getMemberHistory = this.getMemberHistory.bind(this);
    this.getWaiverWire = this.getWaiverWire.bind(this);
    this.addToWaiverWire = this.addToWaiverWire.bind(this);
    this.removeFromWaiverWire = this.removeFromWaiverWire.bind(this);
    this.formatRosterEntries = this.formatRosterEntries.bind(this);
}
```

2.4 Express Route Ordering Fixes

2.4.1 Problem

Dynamic routes with `:id` parameters were defined before static routes, causing Express to incorrectly match paths like `/groups` as an `:id` value.

2.4.2 Solution

File: `backend/routes/roster.js`

Reordered routes so static paths come before dynamic `:id` routes:

```
// CORRECT ORDER:
1. GET  /api/roster           (static)
2. GET  /api/roster/on-deck   (static)
3. GET  /api/roster/search    (static)
4. PUT  /api/roster/reorder   (static)
5. GET  /api/roster/groups    (static)
6. POST /api/roster/groups    (static)
7. GET  /api/roster/waiver-wire (static)
8. DELETE /:id                (dynamic - LAST)
```

```
9. PUT  /:id/on-deck          (dynamic - LAST)
10. GET /:id/profile          (dynamic - LAST)
```

File: `backend/routes/courses.js`
Similar reordering:

```
// CORRECT ORDER:
1. GET  /search     (static)
2. GET  /nearby     (static)
3. GET  /popular    (static)
4. GET  /favorites  (static - before :id)
5. POST /favorites  (static)
6. DELETE /favorites/:id
7. GET  /:id        (dynamic - LAST)
```

2.5 API Endpoint Configuration Fix

2.5.1 Problem

The courses favorites endpoint was misconfigured between frontend and backend.

2.5.2 Solution

File: `TTSGolfApp/config/api.ts`

```
// Changed from
COURSES_FAVORITES: `${API_URL}/api/courses/user/favorites`
// To
COURSES_FAVORITES: `${API_URL}/api/courses/favorites`
```

3 Files Modified

3.1 Backend Files

- `backend/controllers/inviteController.js` - Added constructor bindings, fixed Twilio init
- `backend/controllers/teeTimeController.js` - Added constructor bindings
- `backend/controllers/rosterController.js` - Added constructor bindings
- `backend/repositories/inviteRepository.js` - Fixed column reference
- `backend/routes/roster.js` - Reordered routes
- `backend/routes/courses.js` - Reordered routes

3.2 Frontend Files

- `TTSGolfApp/screens/LoginScreen.tsx` - Added ScrollView for keyboard handling
- `TTSGolfApp/theme/styles.ts` - Added cardScrollContent style
- `TTSGolfApp/config/api.ts` - Fixed favorites endpoint
- `TTSGolfApp/services/api.ts` - Updated to use correct endpoint

4 Technical Notes

4.1 Why Method Binding is Required

When using class-based controllers in Express.js, methods passed as route handlers lose their `this` context. This is because JavaScript's `this` is determined by how a function is called, not where it's defined.

```
// Without binding - 'this' is undefined
router.get("/", controller.getItems);

// With binding in constructor - 'this' refers to controller
this.getItems = this.getItems.bind(this);
```

4.2 Express Route Matching Order

Express matches routes in the order they are defined. A route like `/:id` will match ANY path segment, including static paths like `/groups`. Always define static routes before dynamic parameter routes.

4.3 KeyboardAvoidingView Best Practices

For iOS, use `behavior="padding"`. For Android, use `behavior="height"`. Wrap scrollable content in a `ScrollView` with `keyboardShouldPersistTaps="handled"` to allow button taps while keyboard is open.

5 Testing Recommendations

1. Test OTP login flow on both iOS and Android devices
2. Verify all roster operations (add, remove, reorder, groups)
3. Test friend request send/accept/decline flow
4. Verify course search and favorites functionality
5. Test invite creation with both app users and SMS invites
6. Confirm keyboard behavior on login screen OTP input

6 Conclusion

This session successfully resolved multiple critical API errors that were preventing core functionality from working. The fixes ensure proper method context in controllers, correct route matching in Express, and improved UX for the OTP verification flow. All changes maintain backward compatibility and follow established patterns in the codebase.

Document generated: December 14, 2024